

⚠ Please do not reach out to reviewers directly; messaging them will not speed up your review.

Writing Tests

The `tests/test_outputs.py` file contains pytest tests that verify task completion. Good tests are the foundation of a quality task.

All verifier tests must be written in Python and run with pytest, regardless of the task's implementation language. For non-Python tasks, write Python tests that exercise the CLI, service, files, or processes under test.

`tests/test.sh` is a bash entry point, but it should invoke the Python pytest suite rather than delegating to another language-specific test framework.

Getting Started

Video Tutorial

Creating Tests for Your Task



1.2x

6 min ⚡ 5 min 20 sec

What You'll Learn

- Structure of test_outputs.py
- Writing effective test cases
- Matching tests to task requirements
- Common testing patterns

Basic Structure

PYTHON

 Copy

```
"""Tests for the data processing task."""
import pytest
import json
from pathlib import Path

def test_output_file_exists():
    """Verify the output file was created."""
    assert Path("/output/result.json").exists()

def test_output_format():
    """Verify the output has correct JSON structure."""
    with open("/output/result.json") as f:
        data = json.load(f)

    assert "status" in data
    assert "items" in data
    assert isinstance(data["items"], list)

def test_correct_count():
    """Verify the item count is correct."""
    with open("/output/result.json") as f:
        data = json.load(f)

    assert len(data["items"]) == 42
```

Key Principles

1. Test Behavior, Not Implementation

Run the code and check results. Don't parse source code looking for patterns.

Good:

PYTHON

 Copy

```
def test_function_handles_empty_input():
    """Empty input should return empty list."""
    from app.main import process
    result = process("")
    assert result == []
```

Bad:

PYTHON

 Copy

```
def test_has_empty_check():
    """Check if code has empty input handling."""
    source = open("/app/main.py").read()
    assert "if not" in source # Brittle!
```

2. Informative Docstrings

Every test must have a docstring explaining what behavior it checks. This is validated by CI.

PYTHON

 Copy

```
def test_api_returns_json():
    """API endpoint should return valid JSON with Content-Type header"""
    response = requests.get("http://localhost:8080/api/data")
```

```
assert response.headers["Content-Type"] == "application/json"
assert response.json() # Parseable JSON
```

3. Match Task Requirements

Tests need to fully cover all aspects of the prompt (instruction.md). This includes:

-  All explicit requirements from the prompt
-  Implicitly expected behavior
-  Critical edge cases

Every requirement in the prompt must map to a test. If it is implied or stated in the prompt but not covered by tests, that is a miss.

INSTRUCTION.MD SAYS...	TEST VERIFIES...
"Return empty list for empty input"	<code>test_empty_input_returns_empty_list</code>
"Output to /data/result.csv"	<code>test_output_file_exists</code>
"Include header row"	<code>test_csv_has_header</code>

Here's a very simplified example for demonstrative purposes only. If the prompt is:

Write a Python function called `divide` that takes two numbers and returns the result as a float.

Explicit tests:

PYTHON

 Copy

```
def test_function_exists():
    """A function called 'divide' exists"""

def test_takes_two_numbers():
    """It accepts two numbers as input"""

def test_divides_correctly():
    """It returns the correct division result"""

def test_returns_float():
    """The return type is a float"""
```

Implicit test / edge case:

PYTHON Copy

```
def test_division_by_zero():
    """It handles division by zero"""
```

The prompt never mentions division by zero, but any reasonable person reading "a function that divides two numbers" would expect it to handle that case.

4. Cover Edge Cases

Test the boundaries, not just the happy path:

PYTHON Copy

```
def test_empty_input():
    """Empty input is handled gracefully."""
    assert process("") == []

def test_single_item():
    """Single item input works correctly."""
    assert process("a") == ["a"]

def test_large_input():
```

```
"""Large input is handled efficiently."""
result = process("x" * 10000)
assert len(result) == 10000

def test_special_characters():
    """Special characters are preserved."""
    assert process("hélo  ") == ["hélo", "  "]
```

tests/test.sh

The test runner script sets up the verifier command, runs Python pytest against the test file, and produces a reward file. Do not replace pytest with another test framework such as JUnit, Jest, or [go test](#); use Python pytest tests to drive and validate those systems when needed. It must not install packages or fetch anything from the network at runtime. Bake pytest, plugins, browser drivers, wheels, npm packages, and any other verifier dependencies into the Docker image instead.

BASH Copy

```
#!/bin/bash
set -uo pipefail

# Check if we're in a valid working directory
if [ "$PWD" = "/" ]; then
    echo "Error: No working directory set. Please set a WORKDIR in
    mkdir -p /logs/verifier
    echo 0 > /logs/verifier/reward.txt
    exit 0
fi

mkdir -p /logs/verifier

# pytest and pytest-json-ctrf must be pre-installed in the Docker i
python -m pytest --ctrf /logs/verifier/ctrf.json /tests/test_output
rc=$?

# Produce reward file (REQUIRED)
if [ "$rc" -eq 0 ]; then
    echo 1 > /logs/verifier/reward.txt
```

```
else
    echo 0 > /logs/verifier/reward.txt
fi
```

Note: Test dependencies must be installed in the Dockerfile, NOT in `tests/test.sh`. `tests/test.sh` should not use `uvx`, `pip install`, `npm install`, `curl`, `wget`, `git clone`, or other networked setup commands. Local-only installs from preloaded wheels, such as `pip install --no-index -f /opt/wheels pytest==8.4.1`, are acceptable when needed.

On the reward block and exit codes: The `if [ ... -eq 0 ] ... fi` reward block is the **canonical end of** `test.sh` (using either `$?` inline or a variable like `rc=$?` captured immediately after `pytest` — `check_test_sh` accepts both shapes). No trailing `exit` statement is required or desired after it. Harbor determines pass/fail by reading `/logs/verifier/reward.txt`, **not** the script's exit code — when `pytest` fails, the `else` branch writes `0` and the platform records a failure regardless of the script's own exit status. Reviewers must **not** flag the absence of a trailing `exit` as a defect. The `check_test_sh` static gate enforces this canonical shape, so adding `exit $?` after `fi` will actually fail CI.

Common Patterns

Testing File Output

PYTHON

 Copy

```
def test_csv_output():
    """Verify CSV output format and content."""
    import csv

    with open("/output/data.csv") as f:
```

```
        reader = csv.DictReader(f)
        rows = list(reader)

    assert len(rows) > 0
    assert "id" in rows[0]
    assert "name" in rows[0]
```

Testing API Endpoints

PYTHON

 Copy

```
import requests

def test_health_endpoint():
    """Health check endpoint returns 200."""
    response = requests.get("http://localhost:8080/health")
    assert response.status_code == 200

def test_api_error_handling():
    """Invalid requests return 400."""
    response = requests.post(
        "http://localhost:8080/api/data",
        json={"invalid": "data"}
    )
    assert response.status_code == 400
```

Testing Database State

PYTHON

 Copy

```
import sqlite3

def test_database_populated():
    """Database contains expected records."""
    conn = sqlite3.connect("/app/data.db")
    cursor = conn.execute("SELECT COUNT(*) FROM users")
    count = cursor.fetchone()[0]
    conn.close()
```

```
assert count == 100
```

Testing Command Output

PYTHON

 Copy

```
import subprocess

def test_cli_help():
    """CLI shows help message."""
    result = subprocess.run(
        ["python", "/app/cli.py", "--help"],
        capture_output=True,
        text=True
    )
    assert result.returncode == 0
    assert "Usage:" in result.stdout
```

Anti-Patterns to Avoid

Brittle String Matching

PYTHON

 Copy

```
# BAD: Exact string match
def test_output():
    output = open("/output/log.txt").read()
    assert output == "Processing complete\n"

# GOOD: Check for key content
def test_output():
    output = open("/output/log.txt").read()
    assert "complete" in output.lower()
```

Hardcoded Random Values

PYTHON

 Copy

```
# BAD: Assumes specific random output
def test_random():
    result = generate_random()
    assert result == 42

# GOOD: Check properties
def test_random():
    result = generate_random()
    assert 1 <= result <= 100
```

Order-Dependent Tests

PYTHON

 Copy

```
# BAD: Tests depend on execution order
def test_1_setup():
    global data
    data = load_data()

def test_2_process():
    process(data) # Fails if test_1 didn't run first

# GOOD: Each test is independent
def test_process():
    data = load_data()
    result = process(data)
    assert result is not None
```

CI Validation

Your tests will be validated by:

CHECK	DESCRIPTION
behavior_in_tests	All task requirements have tests

CHECK	DESCRIPTION
behavior_in_task_description	All tested behavior is in instruction.md
informative_test_docstrings	Each test has a docstring
ruff	Code passes linting

Next Steps

- [Run oracle agent](#)
- [Review CI checks](#)

PREVIOUS

[Writing Oracle Solution](#)

NEXT

[Oracle Agent](#)